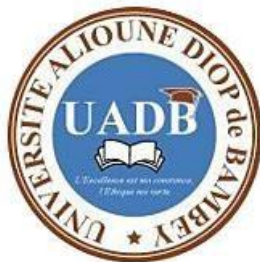


REPUBLIQUE DU SENEGAL

Un peuple Un But Une Foi

Ministère de l'Enseignement Supérieur de la Recherche et de l'Innovation (MESRI)

UNIVERSITE ALIOUNE DIOP DE BAMBEY



UFR : SCIENCES APPLIQUÉES ET TECHNOLOGIES DE L'INFORMATION ET DE LA COMMUNICATION(SATIC)

FILIERE : SYSTÈMES ET RÉSEAUX (SR)

NIVEAU : MASTER 1

Contrôle des versions logicielles avec Git

Présenté par :

Mohamadou Sabaly

Professeur :

Dr. Babou

Année académique 2025-2026

Table des matières

Introduction.....	2
Partie 2 — Initialisation de Git	2
1. Configuration de l'identité	2
2. Création du dépôt	2
Partie 3 — Mise en scène et validation d'un fichier.....	4
3. Création du fichier	4
4. Staging et commit	5
Partie 4 — Modification du fichier et suivi des changements.....	6
5. Modification et nouveau commit	6
6. Comparaison des commits avec git diff	7
Partie 5 — Branches et fusion (Merge).....	8
7. Création et bascule vers une branche	8
8. Modification dans la branche feature	8
9. Fusion de la branche dans master	9
10. Suppression de la branche	9
Partie 6 — Gestion des conflits de fusion.....	10
11. Création du conflit	10
12. Tentative de fusion et détection du conflit	11
13. Résolution manuelle avec vim	11
Partie 7 — Intégration de Git avec GitHub.....	14
14. Création du dépôt GitHub	14
15. Préparation du dossier local	14
16. Liaison avec GitHub et push	15
Conclusion	16

Introduction

Ce rapport présente les travaux pratiques réalisés sur Git, un système de contrôle de version distribué largement utilisé dans le monde du développement logiciel. L'objectif principal était de se familiariser avec les concepts fondamentaux de Git : initialisation d'un dépôt, gestion des commits, travail avec les branches, résolution de conflits, et enfin intégration avec GitHub.

Ce TP a été réalisé sur la machine virtuelle DEVASC, un environnement Linux dédié aux exercices de développement et d'automatisation réseau.

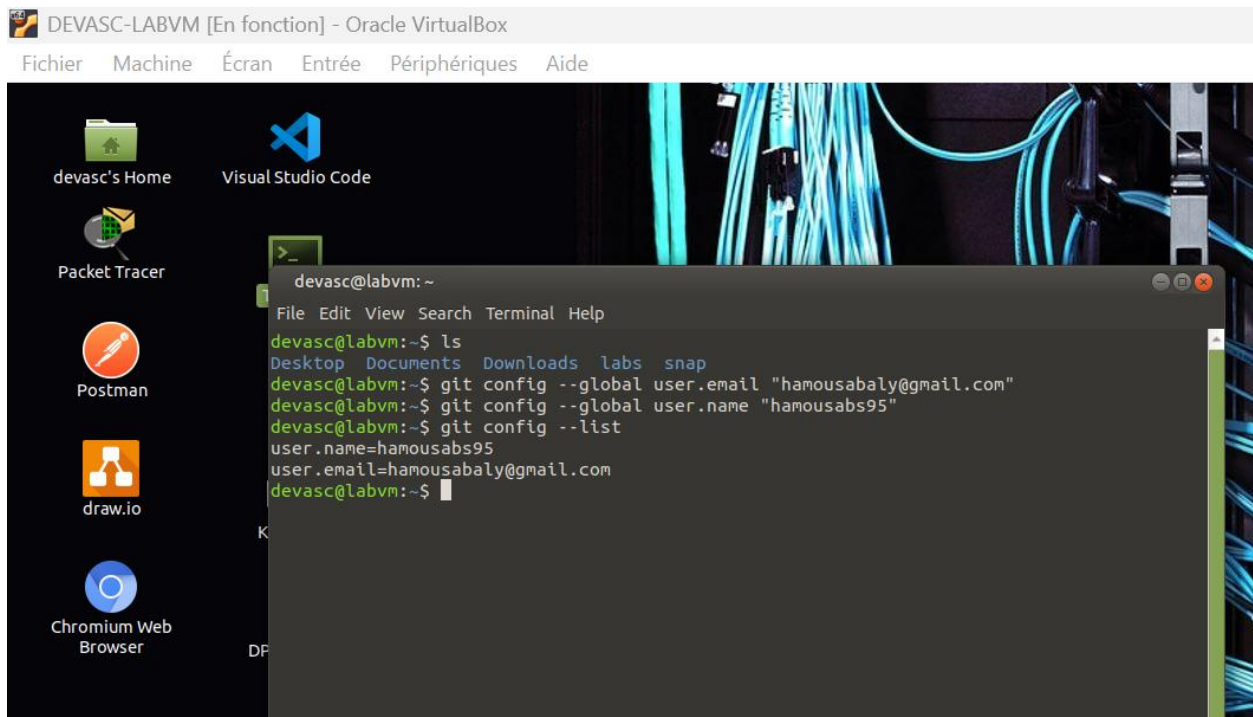
Partie 2 — Initialisation de Git

La première étape consiste à configurer Git avec notre identité et à créer un premier dépôt local.

1. Configuration de l'identité

Avant de commencer à travailler avec Git, il faut lui indiquer qui nous sommes. Ces informations apparaîtront dans chaque commit réalisé.

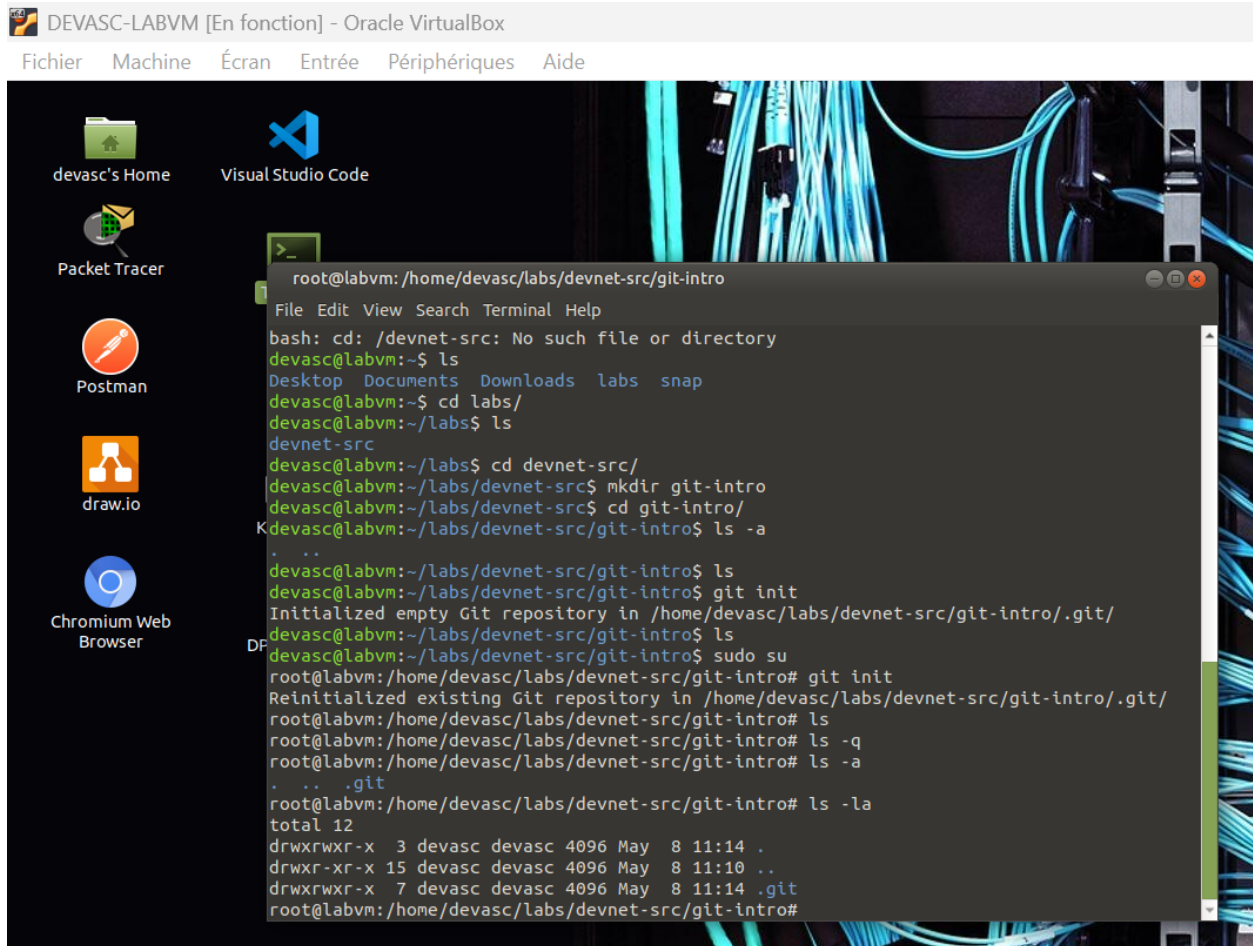
```
git config --global user.name "hamousabs95"  
git config --global user.email "hamousabaly@gmail.com"  
git config --list
```



2. Création du dépôt

On accède au répertoire de travail, on crée un dossier dédié, puis on initialise le dépôt Git.

```
cd ~/devnet-src
mkdir git-intro && cd git-intro
git init
ls -a      # pour voir le dossier caché .git
git status
```



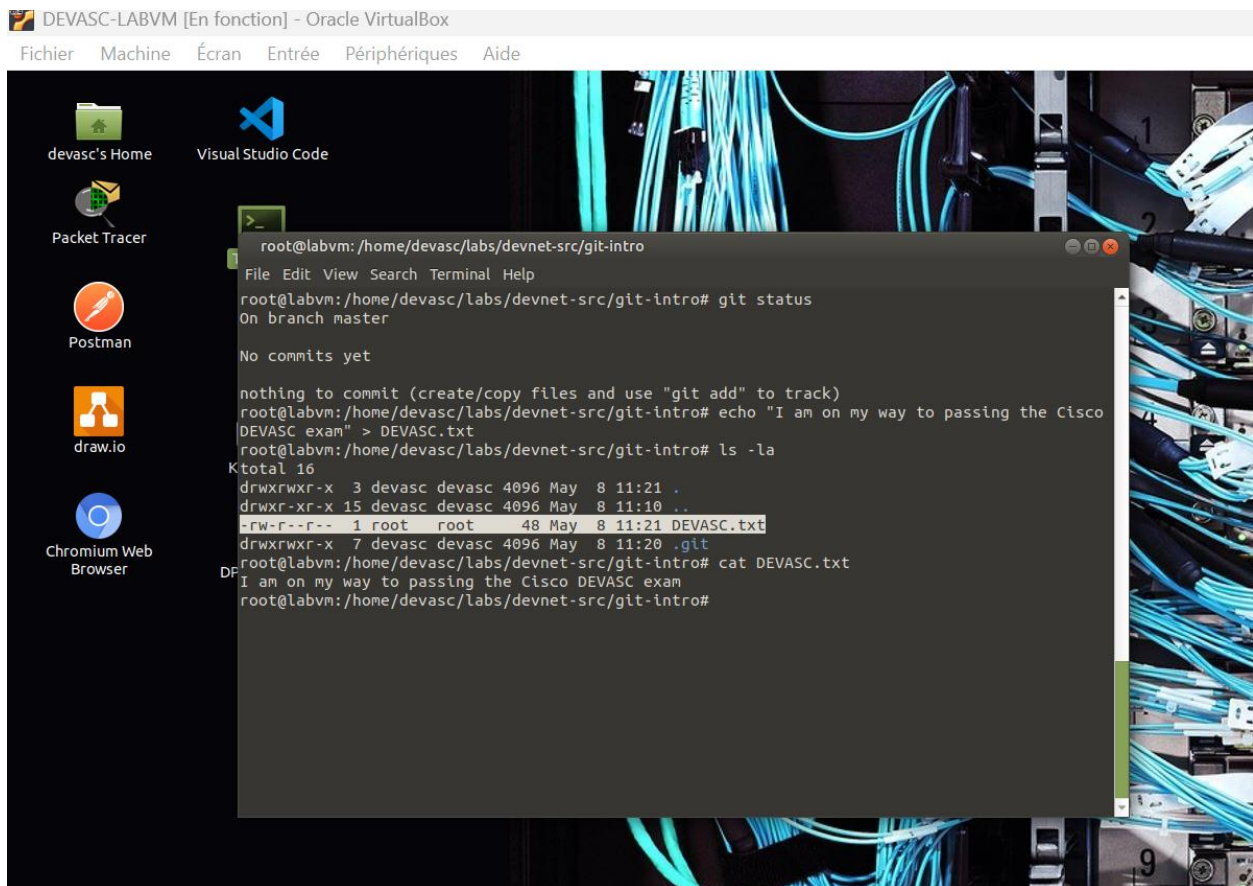
Observation : Git confirme que le dépôt est initialisé et qu'il n'y a rien à valider pour le moment.

Partie 3 — Mise en scène et validation d'un fichier

Maintenant que le dépôt existe, on va y ajouter un premier fichier, le mettre en scène (staging) puis le valider (commit).

3. Création du fichier

```
echo "I am on my way to passing the Cisco DEVASC exam" > DEVASC.txt
ls -la
cat DEVASC.txt
```



```
root@labvm:/home/devasc/labs/devnet-src/git-intro# git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  DEVASC.txt

nothing added to commit but untracked files present_ (use "git add" to track)
```

4. Staging et commit

Le staging est une étape intermédiaire qui permet de préparer les fichiers avant de les valider définitivement dans l'historique Git.

```
git status      # fichier non suivi détecté
git add DEVASC.txt # mise en scène
git status      # affiche 'new file: DEVASC.txt'
git commit -m "Committing DEVASC.txt to begin tracking changes"
git log         # vérifier l'historique
```

```
1 file changed, 1 insertion(+)
create mode 100644 DEVASC.txt
root@labvm:/home/devasc/labs/devnet-src/git-intro# git log
commit b51c0179d350a441ddfc47f499b547b71e53bc2e (HEAD -> master)
Author: root <root@labvm.vm>
Date: Fri May 8 11:32:32 2026 +0000

    Committing DEVASC.txt to begin tracking changes
root@labvm:/home/devasc/labs/devnet-src/git-intro#
```

Observation : Chaque commit est identifié par un hachage SHA1 unique. Les 7 premiers caractères forment l'ID de commit.

Partie 4 — Modification du fichier et suivi des changements

Git permet de suivre précisément toutes les modifications apportées aux fichiers. Dans cette partie, on modifie le fichier existant et on trace les changements.

5. Modification et nouveau commit

```
echo "I am beginning to understand Git!" >> DEVASC.txt
cat DEVASC.txt
git status
git add DEVASC.txt
git commit -m "Added a second line to DEVASC.txt"
```

```
root@labvm:/home/devasc/labs/devnet-src/git-intro
File Edit View Search Terminal Help
root@labvm:/home/devasc/labs/devnet-src/git-intro# echo "I am beginning to understand Git!" >> DEVASC.txt
root@labvm:/home/devasc/labs/devnet-src/git-intro# cat DEVASC.txt
I am on my way to passing the Cisco DEVASC exam
I am beginning to understand Git!
root@labvm:/home/devasc/labs/devnet-src/git-intro# git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   DEVASC.txt

no changes added to commit (use "git add" and/or "git commit -a")
root@labvm:/home/devasc/labs/devnet-src/git-intro# git add DEVASC.txt
root@labvm:/home/devasc/labs/devnet-src/git-intro# git commit -m "Added a second line to DEVASC.txt"
[master 19cf3a2] Added a second line to DEVASC.txt
Committer: root <root@labvm.vm>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

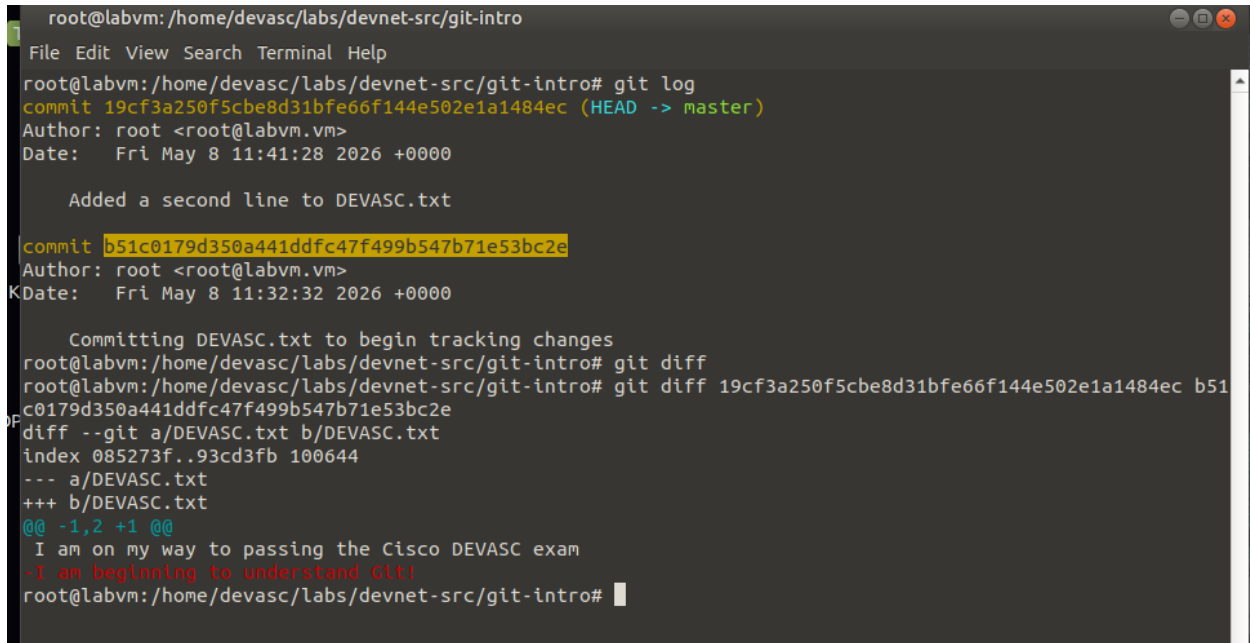
    git commit --amend --reset-author

1 file changed, 1 insertion(+)
root@labvm:/home/devasc/labs/devnet-src/git-intro#
```

6. Comparaison des commits avec git diff

La commande git diff permet de comparer deux versions d'un fichier entre deux commits.

```
git log
git diff 19cf3a250f5cbe8d31bfe66f144e502e1a1484ec b51c0179d350a441ddfc47f499b547b71e53bc2e
```



```
root@labvm: /home/devasc/labs/devnet-src/git-intro
File Edit View Search Terminal Help
root@labvm: /home/devasc/labs/devnet-src/git-intro# git log
commit 19cf3a250f5cbe8d31bfe66f144e502e1a1484ec (HEAD -> master)
Author: root <root@labvm.vm>
Date:   Fri May 8 11:41:28 2026 +0000

    Added a second line to DEVASC.txt

commit b51c0179d350a441ddfc47f499b547b71e53bc2e
Author: root <root@labvm.vm>
Date:   Fri May 8 11:32:32 2026 +0000

    Committing DEVASC.txt to begin tracking changes
root@labvm: /home/devasc/labs/devnet-src/git-intro# git diff
root@labvm: /home/devasc/labs/devnet-src/git-intro# git diff 19cf3a250f5cbe8d31bfe66f144e502e1a1484ec b51c0179d350a441ddfc47f499b547b71e53bc2e
diff --git a/DEVASC.txt b/DEVASC.txt
index 085273f..93cd3fb 100644
--- a/DEVASC.txt
+++ b/DEVASC.txt
@@ -1,2 +1 @@
 I am on my way to passing the Cisco DEVASC exam
-I am beginning to understand git!
root@labvm: /home/devasc/labs/devnet-src/git-intro#
```

Observation : Le signe '+' indique les lignes ajoutées. Le signe '-' indique les lignes supprimées.

Partie 5 — Branches et fusion (Merge)

Les branches permettent de travailler sur des fonctionnalités en parallèle sans toucher à la branche principale (master). Une fois le travail terminé, on fusionne les changements.

7. Création et bascule vers une branche

```
git branch feature      # créer la branche
git branch              # lister les branches (* = active)
git checkout feature    # basculer vers la branche feature
git branch              # vérifier que * est sur feature
```

```
root@labvm: /home/devasc/labs/devnet-src/git-intro
File Edit View Search Terminal Help
root@labvm: /home/devasc/labs/devnet-src/git-intro# git branch feature
root@labvm: /home/devasc/labs/devnet-src/git-intro# git branch
feature
* master
root@labvm: /home/devasc/labs/devnet-src/git-intro# git checkout feature
Switched to branch 'feature'
root@labvm: /home/devasc/labs/devnet-src/git-intro# git branch
* feature
master
```

8. Modification dans la branche feature

```
echo "This text was added originally while in the feature branch" >> DEVASC.txt
cat DEVASC.txt
git add DEVASC.txt
git commit -m "Added a third line in feature branch"
git log
```

```
root@labvm: /home/devasc/labs/devnet-src/git-intro# echo "This text was added originally while in the fea
ture branch" >> DEVASC.txt
root@labvm: /home/devasc/labs/devnet-src/git-intro# cat DEVASC.txt
I am on my way to passing the Cisco DEVASC exam
I am beginning to understand Git!
KThis text was added originally while in the feature branch
root@labvm: /home/devasc/labs/devnet-src/git-intro# git commit -m "Added a third line in feature branch"
On branch feature
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   DEVASC.txt
no changes added to commit (use "git add" and/or "git commit -a")
root@labvm: /home/devasc/labs/devnet-src/git-intro# git log
commit 19cf3a250f5cbe8d31bfe66f144e502e1a1484ec (HEAD -> feature, master)
Author: root <root@labvm.vm>
Date:   Fri May 8 11:41:28 2026 +0000

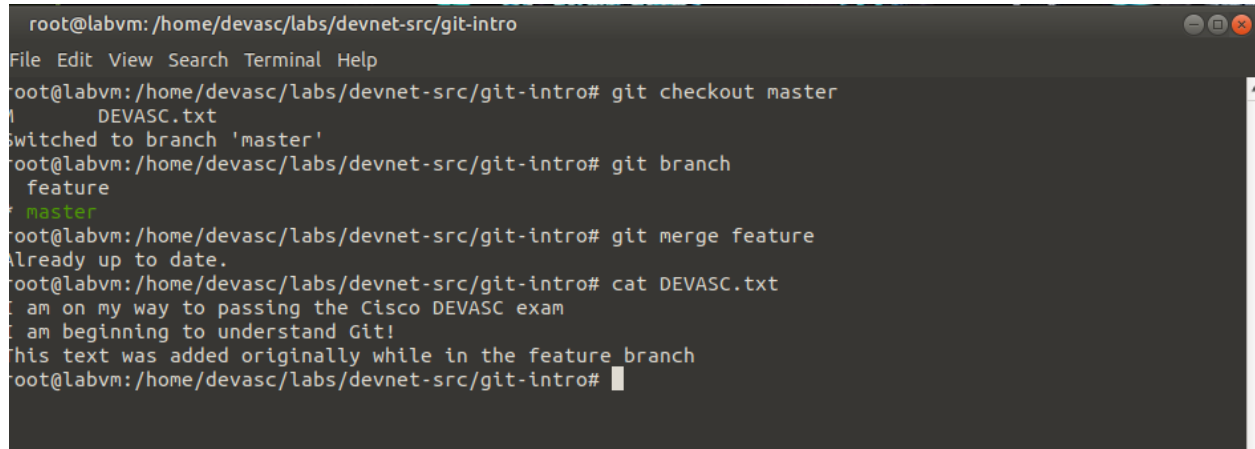
    Added a second line to DEVASC.txt

commit b51c0179d350a441ddfc47f499b547b71e53bc2e
Author: root <root@labvm.vm>
Date:   Fri May 8 11:32:32 2026 +0000

    Committing DEVASC.txt to begin tracking changes
root@labvm: /home/devasc/labs/devnet-src/git-intro#
```

9. Fusion de la branche dans master

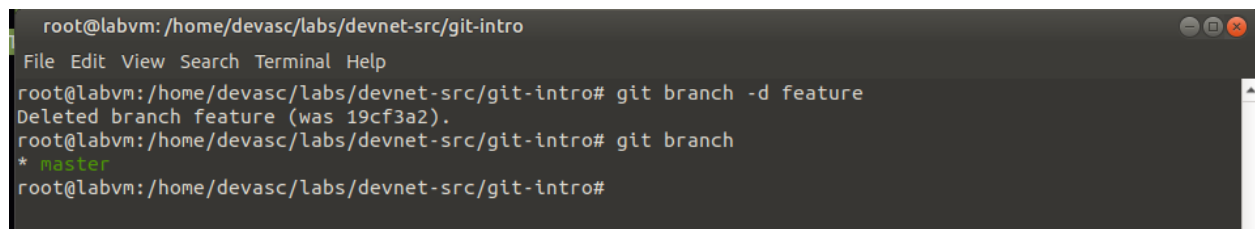
```
git checkout master
git merge feature
cat DEVASC.txt
```

A terminal window titled 'root@labvm: /home/devasc/labs/devnet-src/git-intro'. The terminal shows the following commands and output: 'git checkout master' results in 'switched to branch 'master''; 'git branch' shows 'feature' and '* master'; 'git merge feature' results in 'Already up to date.'; 'cat DEVASC.txt' displays the content of the file: 'I am on my way to passing the Cisco DEVASC exam', 'I am beginning to understand Git!', and 'This text was added originally while in the feature branch'.

```
root@labvm: /home/devasc/labs/devnet-src/git-intro
File Edit View Search Terminal Help
root@labvm:/home/devasc/labs/devnet-src/git-intro# git checkout master
switched to branch 'master'
root@labvm:/home/devasc/labs/devnet-src/git-intro# git branch
feature
* master
root@labvm:/home/devasc/labs/devnet-src/git-intro# git merge feature
Already up to date.
root@labvm:/home/devasc/labs/devnet-src/git-intro# cat DEVASC.txt
I am on my way to passing the Cisco DEVASC exam
I am beginning to understand Git!
This text was added originally while in the feature branch
root@labvm:/home/devasc/labs/devnet-src/git-intro#
```

10. Suppression de la branche

```
git branch -d feature
git branch
```

A terminal window titled 'root@labvm: /home/devasc/labs/devnet-src/git-intro'. The terminal shows the following commands and output: 'git branch -d feature' results in 'Deleted branch feature (was 19cf3a2).'; 'git branch' shows '* master'.

```
root@labvm: /home/devasc/labs/devnet-src/git-intro
File Edit View Search Terminal Help
root@labvm:/home/devasc/labs/devnet-src/git-intro# git branch -d feature
Deleted branch feature (was 19cf3a2).
root@labvm:/home/devasc/labs/devnet-src/git-intro# git branch
* master
root@labvm:/home/devasc/labs/devnet-src/git-intro#
```

Partie 6 — Gestion des conflits de fusion

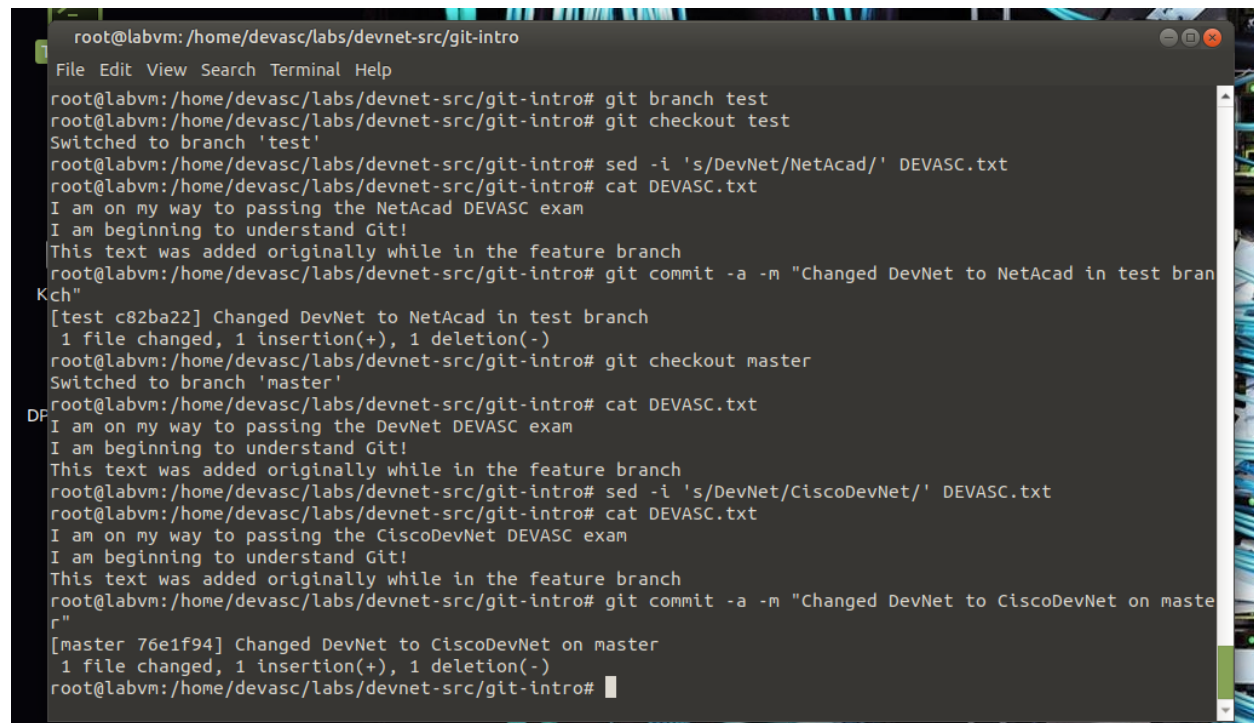
Un conflit de fusion survient quand deux branches ont modifié le même endroit d'un fichier de façon incompatible. Git ne peut pas décider seul quelle version garder — il faut l'intervention manuelle du développeur.

11. Création du conflit

On crée une branche 'test' et on y remplace 'Cisco' par 'NetAcad'. Puis sur master, on remplace 'Cisco' par 'DevNet'. Les deux branches modifient la même ligne.

```
git branch test
git checkout test
sed -i 's/DevNet/NetAcad/' DEVASC.txt
cat DEVASC.txt
git commit -a -m "Changed DevNet to NetAcad in test branch"
```

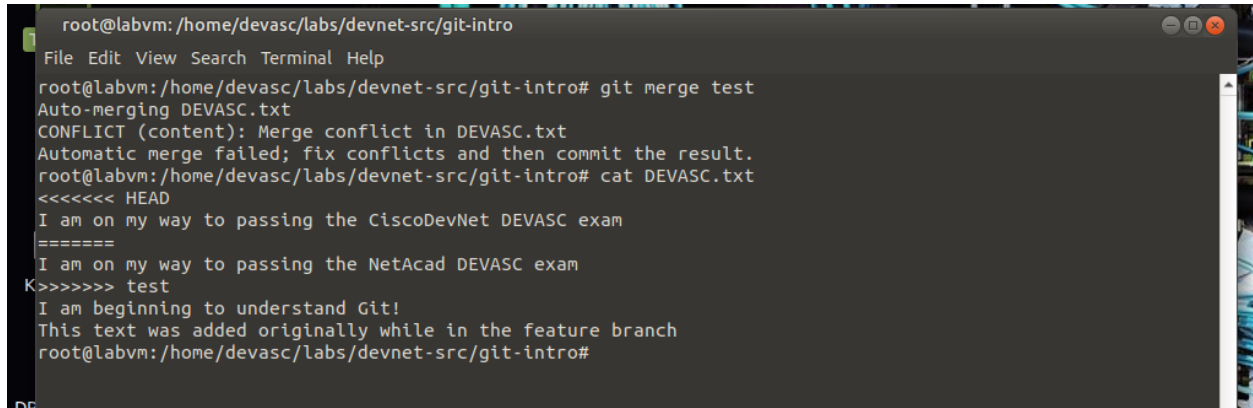
```
git checkout master
sed -i 's/DevNet/CiscoDevNet/' DEVASC.txt
git commit -a -m "Changed DevNet to CiscoDevNet on master"
```



```
root@labvm: /home/devasc/labs/devnet-src/git-intro
File Edit View Search Terminal Help
root@labvm: /home/devasc/labs/devnet-src/git-intro# git branch test
root@labvm: /home/devasc/labs/devnet-src/git-intro# git checkout test
Switched to branch 'test'
root@labvm: /home/devasc/labs/devnet-src/git-intro# sed -i 's/DevNet/NetAcad/' DEVASC.txt
root@labvm: /home/devasc/labs/devnet-src/git-intro# cat DEVASC.txt
I am on my way to passing the NetAcad DEVASC exam
I am beginning to understand Git!
This text was added originally while in the feature branch
root@labvm: /home/devasc/labs/devnet-src/git-intro# git commit -a -m "Changed DevNet to NetAcad in test branch"
[master c82ba22] Changed DevNet to NetAcad in test branch
1 file changed, 1 insertion(+), 1 deletion(-)
root@labvm: /home/devasc/labs/devnet-src/git-intro# git checkout master
Switched to branch 'master'
root@labvm: /home/devasc/labs/devnet-src/git-intro# cat DEVASC.txt
I am on my way to passing the DevNet DEVASC exam
I am beginning to understand Git!
This text was added originally while in the feature branch
root@labvm: /home/devasc/labs/devnet-src/git-intro# sed -i 's/DevNet/CiscoDevNet/' DEVASC.txt
root@labvm: /home/devasc/labs/devnet-src/git-intro# cat DEVASC.txt
I am on my way to passing the CiscoDevNet DEVASC exam
I am beginning to understand Git!
This text was added originally while in the feature branch
root@labvm: /home/devasc/labs/devnet-src/git-intro# git commit -a -m "Changed DevNet to CiscoDevNet on master"
[master 76e1f94] Changed DevNet to CiscoDevNet on master
1 file changed, 1 insertion(+), 1 deletion(-)
root@labvm: /home/devasc/labs/devnet-src/git-intro#
```

12. Tentative de fusion et détection du conflit

```
git merge test      # conflit détecté !  
cat DEVASC.txt      # voir les marqueurs de conflit
```



```
root@labvm: /home/devasc/labs/devnet-src/git-intro  
File Edit View Search Terminal Help  
root@labvm: /home/devasc/labs/devnet-src/git-intro# git merge test  
Auto-merging DEVASC.txt  
CONFLICT (content): Merge conflict in DEVASC.txt  
Automatic merge failed; fix conflicts and then commit the result.  
root@labvm: /home/devasc/labs/devnet-src/git-intro# cat DEVASC.txt  
<<<<<< HEAD  
I am on my way to passing the CiscoDevNet DEVASC exam  
=====  
I am on my way to passing the NetAcad DEVASC exam  
K>>>>>> test  
I am beginning to understand Git!  
This text was added originally while in the feature branch  
root@labvm: /home/devasc/labs/devnet-src/git-intro#
```

NB : Lors de la réalisation de cette partie, j'ai quitté le terminal par erreur avant de pouvoir capturer l'étape **git merge test**. En relançant la commande, Git affichait "**Already up to date**" car les deux branches avaient déjà le même contenu, ce qui empêchait la création d'un conflit réel.

Pour remédier à cela, j'ai dû recréer manuellement la situation de conflit en repartant d'un état antérieur du dépôt avec **git reset --hard**, en recréant la branche **test** sur le bon commit, puis en modifiant la même ligne différemment dans chaque branche (**NetAcad** dans **test**, **CiscoDevNet** dans **master**).

Cette erreur m'a permis de mieux comprendre le fonctionnement interne de Git, notamment la notion d'historique de commits et l'utilisation de **git reset** pour naviguer dans cet historique.

13. Résolution manuelle avec vim

On ouvre le fichier avec vim, on supprime les lignes en conflit (marqueurs et la version à rejeter) pour ne garder que la version souhaitée.

```
vim DEVASC.txt
```

[illegible]

Dans vim : utiliser les flèches, taper 'dd' pour supprimer une ligne, puis ESC > :wq > Entrée pour sauvegarder.

```
Cat DEVASC.txt
git commit -a -m "Manually merged from test branch"
git log
```

```
root@labvm: /home/devasc/labs/devnet-src/git-intro
File Edit View Search Terminal Help

commit 692603c35a1fe70f3d8869c196f5adfa84eea6e7 (HEAD -> master)
Merge: 76e1f94 c82ba22
Author: hamousabs95 <hamousabaly@gmail.com>
Date: Fri May 8 14:49:14 2026 +0000

    Manually merged from test branch

commit 76e1f9414c6d6e9c1b34377cd35e6fdef4e5bbd5
Author: hamousabs95 <hamousabaly@gmail.com>
Date: Fri May 8 13:26:12 2026 +0000

    Changed DevNet to CiscoDevNet on master

commit c82ba227a91975ef6b0c302662fd3a227e456056 (test)
Author: hamousabs95 <hamousabaly@gmail.com>
Date: Fri May 8 13:24:09 2026 +0000

    Changed DevNet to NetAcad in test branch

commit c694734eb453a43514058462bb48f8460ecd4b43
Author: root <root@labvm.vm>
Date: Fri May 8 13:11:11 2026 +0000

    Changed Cisco to DevNet

commit 121d3507b64ae07507bd6343966d1dd122fb41b2
Author: root <root@labvm.vm>
Date: Fri May 8 13:08:23 2026 +0000
```

Observation : La résolution de conflits est une compétence essentielle dans un travail collaboratif. Git marque clairement les zones conflictuelles pour faciliter la prise de décision.

Partie 7 — Intégration de Git avec GitHub

GitHub est une plateforme cloud permettant d'héberger des dépôts Git et de collaborer à plusieurs sur un même projet. Dans cette partie, on pousse notre dépôt local vers GitHub.

14. Création du dépôt GitHub

Sur github.com, on crée un nouveau dépôt avec les paramètres suivants :

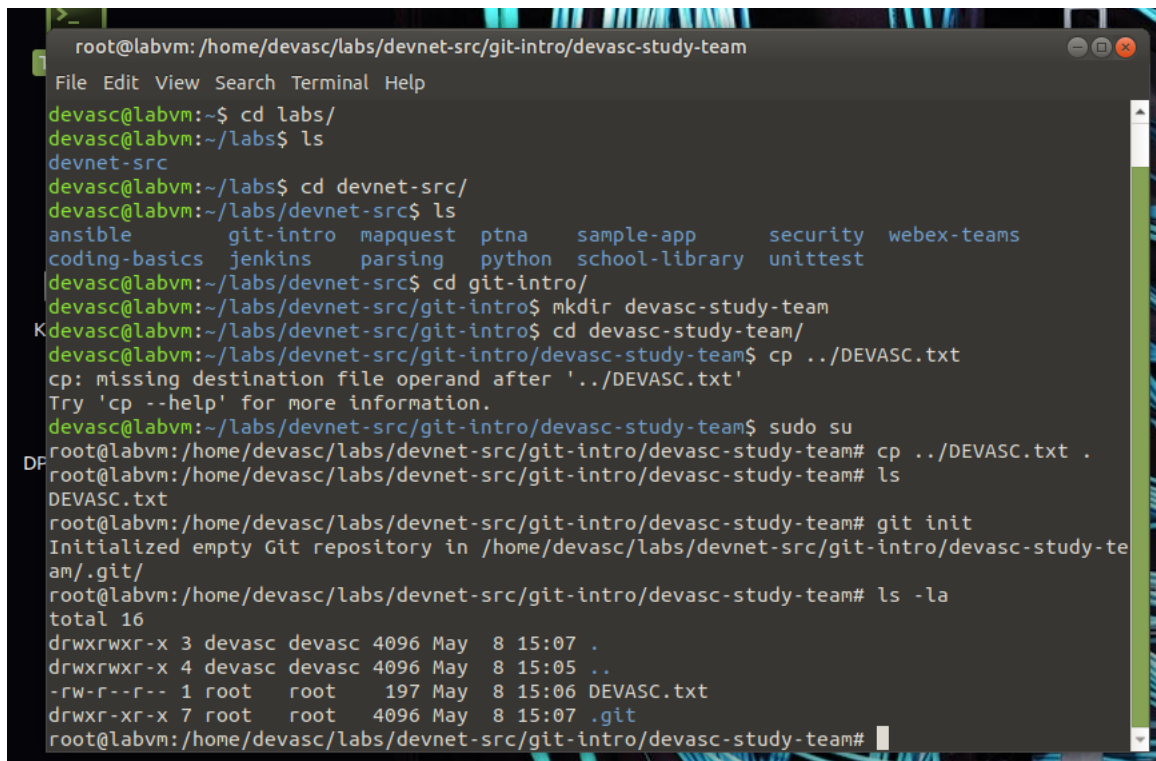
Nom : devasc-study-team

Description : Travailler ensemble pour réussir l'examen DEVASC

Visibilité : Private

15. Préparation du dossier local

```
mkdir devasc-study-team
cd devasc-study-team
cp ../DEVASC.txt .
ls && cat DEVASC.txt
git init
```



```
root@labvm: /home/devasc/labs/devnet-src/git-intro/devasc-study-team
File Edit View Search Terminal Help
devasc@labvm:~$ cd labs/
devasc@labvm:~/labs$ ls
devnet-src
devasc@labvm:~/labs$ cd devnet-src/
devasc@labvm:~/labs/devnet-src$ ls
ansible          git-intro  mapquest  ptna      sample-app  security  webex-teams
coding-basics    jenkins   parsing   python    school-library  unittest
devasc@labvm:~/labs/devnet-src$ cd git-intro/
devasc@labvm:~/labs/devnet-src/git-intro$ mkdir devasc-study-team
Kdevasc@labvm:~/labs/devnet-src/git-intro$ cd devasc-study-team/
devasc@labvm:~/labs/devnet-src/git-intro/devasc-study-team$ cp ../DEVASC.txt
cp: missing destination file operand after '../DEVASC.txt'
Try 'cp --help' for more information.
devasc@labvm:~/labs/devnet-src/git-intro/devasc-study-team$ sudo su
DProot@labvm:/home/devasc/labs/devnet-src/git-intro/devasc-study-team# cp ../DEVASC.txt .
root@labvm:/home/devasc/labs/devnet-src/git-intro/devasc-study-team# ls
DEVASC.txt
root@labvm:/home/devasc/labs/devnet-src/git-intro/devasc-study-team# git init
Initialized empty Git repository in /home/devasc/labs/devnet-src/git-intro/devasc-study-team/.git/
root@labvm:/home/devasc/labs/devnet-src/git-intro/devasc-study-team# ls -la
total 16
drwxrwxr-x 3 devasc devasc 4096 May  8 15:07 .
drwxrwxr-x 4 devasc devasc 4096 May  8 15:05 ..
-rw-r--r-- 1 root   root    197 May  8 15:06 DEVASC.txt
drwxr-xr-x 7 root   root    4096 May  8 15:07 .git
root@labvm:/home/devasc/labs/devnet-src/git-intro/devasc-study-team#
```

16. Liaison avec GitHub et push

```
git config --global user.name "hamousabs95"
git config --global user.email "hamousabaly@gmail.com"
git remote add origin https://github.com/hamousabs95/devasc-study-team.git
git remote -v
git add DEVASC.txt
git commit -m "Added DEVASC.txt to devasc-study-team repo"
git log
git status
git push origin master
```

```
root@labvm: /home/devasc/labs/devnet-src/git-intro/devasc-study-team
File Edit View Search Terminal Help
root@labvm: /home/devasc/labs/devnet-src/git-intro/devasc-study-team# git config --global user.name "
hamousabs95"
root@labvm: /home/devasc/labs/devnet-src/git-intro/devasc-study-team# git config --global user.email
"hamousabaly@gmail.com"
root@labvm: /home/devasc/labs/devnet-src/git-intro/devasc-study-team# git remote add origin https://g
ithub.com/hamousabs95/devasc-study-team.git
root@labvm: /home/devasc/labs/devnet-src/git-intro/devasc-study-team# git remote -v
origin https://github.com/hamousabs95/devasc-study-team.git (fetch)
origin https://github.com/hamousabs95/devasc-study-team.git (push)
root@labvm: /home/devasc/labs/devnet-src/git-intro/devasc-study-team# git add DEVASC.txt
root@labvm: /home/devasc/labs/devnet-src/git-intro/devasc-study-team# git commit -m "Added DEVASC.txt
to devasc-study-team repo"
[master (root-commit) fbf5ed6] Added DEVASC.txt to devasc-study-team repo
1 file changed, 4 insertions(+)
create mode 100644 DEVASC.txt
root@labvm: /home/devasc/labs/devnet-src/git-intro/devasc-study-team#
```

```
root@labvm: /home/devasc/labs/devnet-src/git-intro/devasc-study-team
File Edit View Search Terminal Help
root@labvm: /home/devasc/labs/devnet-src/git-intro/devasc-study-team# git push origin master
Username for 'https://github.com': hamousabs95
Password for 'https://hamousabs95@github.com':
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 362 bytes | 51.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0)
To https://github.com/hamousabs95/devasc-study-team.git
* [new branch] master -> master
root@labvm: /home/devasc/labs/devnet-src/git-intro/devasc-study-team# git log
commit fbf5ed6aba42076db3fb49012b64a9be17871037 (HEAD -> master, origin/master)
Author: hamousabs95 <hamousabaly@gmail.com>
Date: Fri May 8 15:14:19 2026 +0000

    Added DEVASC.txt to devasc-study-team repo
root@labvm: /home/devasc/labs/devnet-src/git-intro/devasc-study-team#
```

Conclusion

Ces travaux pratiques ont permis de découvrir et de pratiquer les fonctionnalités essentielles de Git de façon progressive et concrète. Voici ce que j'ai retenu de chaque partie :

Partie	Ce que j'ai appris
Partie 2	Initialiser un dépôt Git avec <code>git init</code> et configurer son identité
Partie 3	Créer un fichier, le mettre en scène avec <code>git add</code> et le valider avec <code>git commit</code>
Partie 4	Suivre les modifications avec <code>git diff</code> et comprendre l'historique avec <code>git log</code>
Partie 5	Travailler en parallèle avec des branches et fusionner le travail avec <code>git merge</code>
Partie 6	Identifier et résoudre manuellement les conflits de fusion
Partie 7	Lier un dépôt local à GitHub et publier son travail avec <code>git push</code>

En conclusion, Git est un outil indispensable pour tout développeur. Il offre un filet de sécurité grâce à l'historique des versions, facilite le travail en équipe grâce aux branches, et permet une collaboration à l'échelle mondiale grâce à des plateformes comme GitHub.